Efficient Strict Serializability for Stateful Serverless Workflows via Workflow Reconciliation

Shenghao Ao
132ash@sjtu.edu.cn
Shanghai Jiao Tong University
Shanghai, China

Zijun Li
lzjzx1122@sjtu.edu.cn
Shanghai Jiao Tong University
Shanghai, China

Chuhao Xu
barrin@sjtu.edu.cn
Shanghai Jiao Tong University
Shanghai, China

Quan Chen
chen-quan@cs.sjtu.edu.cn
Shanghai Jiao Tong University
Shanghai, China

Shuo Quan
quansh@chinatelecom.cn
Cloud Computing Research Institute, China Telecom
Beijing, China

Yanan Yang
yangyn11@chinatelecom.cn
Cloud Computing Research Institute, China Telecom
Beijing, China

Meiqi Zhu
zhumq@chinatelecom.cn
China Telecom Cloud Technology Co., Ltd.
Beijing, China

Yujun Wang
wangyj3@chinatelecom.cn
China Telecom Cloud Technology Co., Ltd.
Beijing, China

Jie Wu
jiewu@temple.edu
Cloud Computing Research Institute, China Telecom
Beijing, China
Temple University
Philadelphia, USA

Minyi Guo
guo-my@cs.sjtu.edu.cn
Shanghai Jiao Tong University
Shanghai, China
Guizhou University
Guiyang, China

Abstract

Serverless computing, represented by Function-as-a-Service (FaaS), is increasingly adopted for deploying complex applications via stateful workflows. However, the lack of strong consistency (i.e., strict serializability) transaction support hinders its broader applicability. Existing solutions are inadequate, either offering only weak consistency or suffering severe performance degradation from concurrency control models that blindly retry requests after conflicts.

We therefore present FaaSRep, a high-performance serverless framework that provides strict serializability through *workflow reconciliation*, which resolves conflicts by learning dependencies from an optimistic execution and using them to guide an organized retry. FaaSRep builds dependencies at validation time, using the validator's global access view without extra execution-path communication. It further uses a hybrid optimistic/pessimistic protocol to preserve correctness when application-level aborts invalidate the learned order. To reduce the cost of reconciliation, FaaSRep re-executes only affected functions and uses placement information from the initial execution to establish a peer-to-peer fast path during retry. Compared with state-of-the-art systems on real-world applications, FaaSRep improves average throughput by 1.44–2.73× over Concord and 3.81–9.01× over Beldi.

CCS Concepts: • **Computer systems organization → Cloud computing**; • **Information systems → Distributed database transactions**.

Keywords: FaaS, Serverless Computing, Serverless Workflow, Transaction, Consistency



This work is licensed under a Creative Commons Attribution 4.0 International License.

SIGOPS ATC 2026, Hong Kong, China

© 2026 Copyright held by the owner/author(s).

ACM ISBN 979-8-4007-2818-1/2026/11
https://doi.org/10.1145/3828523.3857137

ACM Reference Format:
Shenghao Ao, Zijun Li, Chuhao Xu, Quan Chen, Shuo Quan, Yanan Yang, Meiqi Zhu, Yujun Wang, Jie Wu, and Minyi Guo. 2026. Efficient Strict Serializability for Stateful Serverless Workflows via Workflow Reconciliation. In *The 2026 ACM SIGOPS Annual Technical Conference (SIGOPS ATC 2026), November 15–18, 2026, Hong Kong, China.* ACM, New York, NY, USA, 16 pages. https://doi.org/10.1145/3828523.3857137

1 Introduction

Serverless computing, also known as *Function-as-a-Service* (FaaS), has gained widespread adoption in the cloud-native era thanks to its advantages such as ease of use and pay-as-you-go billing [1, 27, 45]. To support complex applications like data analytics pipelines [6, 12, 20] and machine learning inference chains [3, 11], modern serverless platforms have evolved from executing isolated functions to orchestrating multi-step workflows. They commonly represent a workflow as a directed acyclic graph (DAG), where nodes are functions and edges capture invocation order and control dependencies [30, 33, 38]. This workflow model is increasingly used not only for stateless pipelines, but also for stateful workflows whose functions access and update shared external data items, such as orders, inventories, and payments [59, 62].

Such stateful workflows often require transaction semantics to ensure correctness [59, 62]. For example, an online shopping workflow [4, 53] may need to check inventory, deduct stock, and create an order. These state operations should either all take effect or none should. Meanwhile, concurrent requests should not lead to anomalies such as overselling. We refer to such workflows as *transactional serverless workflows*: multiple functions within one workflow jointly form a transaction with strict serializability [9]. Their effects commit or abort atomically, and strict serializability ensures that concurrent workflow requests are equivalent to a real-time-respecting serial execution. Without workflow-level transaction support, developers must manually handle partial effects and concurrent updates across independently scheduled functions [17], placing a heavy burden on them to preserve application correctness.

Existing serverless transaction systems mainly enforce these guarantees through pessimistic or optimistic concurrency control. Figure 1 compares them using a simple example: concurrent requests $R_1, R_2, ..., R_n$ execute the same workflow $f_1 \rightarrow f_2 \rightarrow f_3$. Their f_2 functions update the same shared object and therefore conflict. *Pessimistic* approaches expose conflicts during execution. For example, Beldi [59] uses remote locks, whereas Concord [49] employs a cache-coherence protocol to prevent concurrent conflicting updates. *Optimistic* approaches (OCC) [29] allow requests to execute first and validate their read/write sets later. Although these approaches discover conflicts at different times, they handle a failed attempt in the same way: the conflicted workflow request is discarded and retried from scratch.

This strategy is inefficient because retries compete for the same shared objects again. Such persistent contention can cause an unbounded cycle of conflict, abort, and retry that dominates end-to-end latency. In our experiments, retry overhead increases latency by more than 4× (§2.2).

Our key observation is that an expensive failed attempt can also reveal information that helps with retries. In the example above, once the runtime observes conflicts among

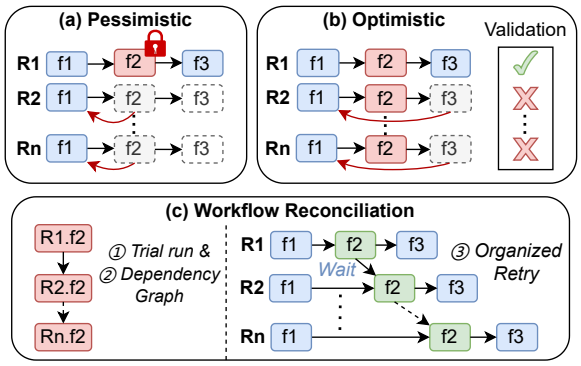


Figure 1. Comparison of conflict handling methods. Pessimistic/optimistic approaches retry failed requests repeatedly, while workflow reconciliation organizes conflicting functions within one retry through the dependency graph.

f_2 instances, it can order the next attempt, for example requiring $R_k.f_2$ to run after $R_{k-1}.f_2$. The retry then follows the learned order instead of repeating competition.

We call this conflict-handling model *workflow reconciliation*. It separates conflict discovery from conflict resolution, as shown in Figure 1 (c). The first execution runs optimistically to reveal which functions from concurrent workflow requests access the same objects. Requests commit normally if no conflict is found. Otherwise, the runtime selects a serialization order and constructs a dependency graph that orders conflicting function instances. The retry follows this graph, serializing conflicting accesses through explicit constraints while allowing independent functions to proceed concurrently. Workflow reconciliation thus turns repeated abort-and-retry cycles into an information-guided retry.

However, efficient implementation of workflow reconciliation in serverless systems is non-trivial. First, dependency construction requires a global view of function-level accesses across concurrent requests, but functions may be placed independently across worker nodes [33, 38]. A naive solution must either query multiple nodes or maintain a centralized access log, introducing coordination overhead on the execution path. Second, application-level aborts [59] can invalidate dependencies inferred from the initial execution. For example, if $R_1.f_2$ in Figure 1 aborts, continuing $R_2.f_2$ under the same dependency order may therefore make it read from an invalid predecessor and then break the serialization order. Third, reconciliation would lose much benefit if retries still pay the same control plane scheduling cost [31, 33] and re-execute the entire workflow. In Figure 1, only f_2 is involved in the conflict, but a naive retry also reruns unaffected functions such as f_1, wasting computation and scheduling work.

In this paper, we present *FaaSRep*, a transactional serverless runtime that realizes workflow reconciliation to provide strict serializability. FaaSRep addresses the above serverless-specific challenges with three techniques. First, it constructs

the dependency graph at validation time. By piggybacking on the stale-read checking step, FaaSRep obtains a global view of concurrent accesses without introducing a separate cross-node graph-construction phase. Second, it uses a hybrid optimistic/pessimistic reconciliation protocol: the common case remains optimistic, while requests affected by application-level aborts switch to a conservative mode when the inferred order becomes unstable. Third, it reduces reconciliation cost through function-level re-execution and a peer-to-peer fast path. During retry, FaaSRep re-executes only dirty functions and necessary dependents, reusing valid function results. Meanwhile, the fast path uses container placement information from the initial execution to let containers communicate directly, reducing repeated control plane scheduling.

We evaluate FaaSRep on an 8-node cluster. Using real-world applications, FaaSRep improves average throughput by 1.44–2.73× over Concord and 3.81–9.01× over Beldi. This paper makes the following contributions:

- We introduce *workflow reconciliation* for transactional serverless workflows. It uses data dependencies revealed by one optimistic execution to guide the retry, rather than retrying blindly as in existing concurrency-control approaches.
- We design a strictly serializable reconciliation protocol for serverless workflows. The protocol constructs function-level dependencies by piggybacking on validation and handles application-level aborts with a hybrid optimistic/pessimistic mode.
- We implement FaaSRep, a transactional serverless runtime that supports function-level re-execution and a peer-to-peer fast path, reducing redundant computation and control plane scheduling during retry.

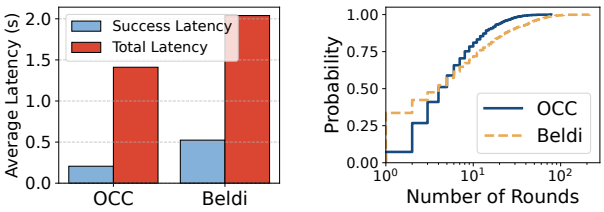
2 Background and Motivation

2.1 Transactional Serverless Workflow

Applications in serverless computing are often structured as workflows of functions and represented as directed acyclic graphs (DAGs). A workflow triggers downstream functions through control plane relays [30]. Because serverless platforms create function instances on demand and place them independently across worker nodes [1, 27, 32], a single workflow request may span multiple nodes [30, 33].

This paper focuses on transactional serverless workflows. Functions in the same workflow request access shared objects through data APIs and jointly form a strictly serializable transaction [9, 21]. Strict serializability ensures concurrent requests produce the same effects as a serial execution that respects real time: if one request commits before another begins, the former must appear earlier in the serial order.

These guarantees preserve application correctness. In an online shopping workflow, with weak consistency, concurrent orders may purchase the same item, causing overselling.



(a) Latency comparison. **(b)** CDF of rounds.

Figure 2. Execution results on Beldi and OCC. Many requests require repeated execution rounds.

Saga [22] exposes intermediate commits, making other orders observe reduced inventory for a purchase whose payment later fails. Its compensation restores inventory but cannot undo those observations. Strict serializability instead ensures subsequent orders observe completed purchases.

The payment failure above also illustrates that workflow requests may abort because of application logic, unlike pure database transactions. Other examples include insufficient inventory or balance [16, 21, 59]. We refer to such aborts as application-level aborts, to distinguish them from conflict-induced aborts triggered by concurrency control.

2.2 Implications of Existing Solutions

Existing serverless transaction systems follow either pessimistic or optimistic concurrency control. Pessimistic systems (Beldi [59], Boki [26], and Concord [49]) coordinate shared-state accesses during execution using locks or coherence protocols. Optimistic concurrency control (OCC) [2, 29, 55] allows requests to execute first, records their read/write sets, and then validates them to detect conflicts. It can also validate requests in batches to reduce network overhead [19].

These designs differ in where they detect conflicts, but they converge after a conflict is found: the failed attempt is discarded and the workflow request is retried, as illustrated by the red arrow in Figure 1. To quantify this cost, we run a travel reservation workload [26, 59] in a closed-loop setting. Figure 2a separates the latency of the final successful attempt from the total end-to-end latency. In OCC and Beldi, the final successful attempt accounts for only 14.6% and 25.7% of the total latency, respectively. Figure 2b further shows that about 26% of requests require more than ten attempts before completion. Thus, the dominant cost is not the execution that commits, but the sequence of failed attempts.

The root cause is that these systems simply re-execute the workflow and again compete with concurrent requests for the same shared objects. If contention persists, the request can fail and retry repeatedly. The limitation also affects the 2PL protocol used by Beldi and Boki, because their deadlock prevention schemes such as Wait-Die [46] still force lower-priority transactions to abort under lock contention.

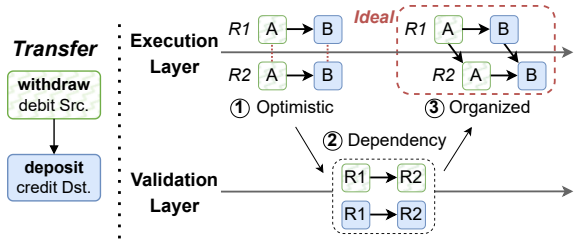


Figure 3. The execution diagram of workflow reconciliation.

2.3 Opportunity of Workflow Reconciliation

2.3.1 Ideal Way: Organized Execution. An ideal conflict-handling model would order conflicting accesses before execution. Knowing which requests access the same objects lets the system schedule one request to update an object before another observes its result. This explicit ordering resolves conflicts without repeated failed attempts.

The red-boxed part of Figure 3 illustrates this ideal process with a transfer workflow. The workflow has two functions: *withdraw* debits the source account, and *deposit* credits the destination account. Suppose two concurrent requests, R_1 and R_2, transfer money between the same accounts A and B. If the system already knows this access pattern and chooses the serial order $R_1 \prec R_2$, it can execute $R_2.withdraw$ after $R_1.withdraw$ and $R_2.deposit$ after $R_1.deposit$. The two requests then complete without racing on the same accounts, while unrelated accesses remain concurrent.

The key difficulty is that this order requires access information before execution. However, serverless functions may access different data for different inputs, rather than a fixed set, making read/write sets difficult to obtain in advance.

2.3.2 Workflow Reconciliation Methodology. Failed executions contain information needed to approximate ideal organized execution: the runtime can collect each function's read/write set for external transactional objects. A failed attempt thus serves as an information-discovery phase.

Therefore, we introduce *workflow reconciliation*, a two-stage conflict-handling model that separates information discovery from ordered execution. In the first stage, concurrent workflow requests execute optimistically and produce function-level metadata. In the second stage, FaaSRep selects a serialization order and uses the observed metadata to construct a dependency graph, through which conflicts are resolved by an organized retry.

Workflow reconciliation is inspired by prior work on dependency-based transaction re-execution [15, 19, 57], and is designed specifically for distributed serverless workflows. Section 8 discusses the differences in detail.

Execution flow. Figure 3 illustrates the execution flow of workflow reconciliation. R_1 and R_2 first execute without coordination and expose their accesses to accounts A and B. The validation layer observes that the two requests conflict on the same accounts, chooses the order $R_1 \prec R_2$, and constructs dependencies from R_1 to R_2 at the corresponding functions. During the retry, R_2 waits only where it depends on R_1, so the conflicting account accesses are serialized.

2.4 Challenges and Solutions

However, designing a workflow reconciliation-based scheme for serverless environments is nontrivial.

2.4.1 Cross-Node Dependency Construction. Building the dependency graph requires a global view of function-level read/write metadata. In serverless platforms, this metadata is naturally distributed across worker nodes because function invocations are scheduled independently and one workflow may span multiple machines. A naive design can collect metadata from workers on demand or maintain a centralized access log during execution, but both add remote communication and shared coordination to the execution path. As a result, they incur overhead similar to remote locking in Beldi [59], where remote communication accounts for more than 25% of total latency (Section 7.7).

Solution: Delayed dependency construction. FaaSRep constructs dependencies at OCC-style validation time, where the validator has a global view of data accesses across concurrent requests. By piggybacking dependency construction on the existing stale-read checking step, FaaSRep obtains the graph with minimal overhead.

2.4.2 Application-level Aborts Invalidate Inferred Orders. The application-level aborts triggered by serverless functions can make a previously valid dependency graph unsafe. Figure 4 shows an example. If R_3 aborts, R_4 loses the predecessor information in the dependency graph and can no longer obtain the latest upstream update V_2 in the serialized order. As a result, R_4 can only read the older committed version V_1 from global storage, thereby violating strict serializability. Rebuilding the graph after each abort is also unsafe as a general strategy, because later application-level aborts may trigger further invalidations and lead to cascading, unbounded restarts [25].

Solution: Hybrid optimistic/pessimistic mode. FaaSRep introduces a pessimistic mode to handle application-level aborts. In this mode, a request is retried only after its necessary dependency has been resolved, and will not suffer another re-execution. To balance correctness and performance, requests first attempt an optimistic retry and fall back to pessimistic mode only when necessary.

2.4.3 Avoid Redundant Work during Retry. Even with a correct dependency graph, reconciliation can lose its advantage if retry follows the standard invocation path. Serverless control planes route function calls and schedule successors after predecessors complete, adding non-negligible indirect communication overhead that can reach 181.3 ms for real-world applications [39]. If each retry step still goes through

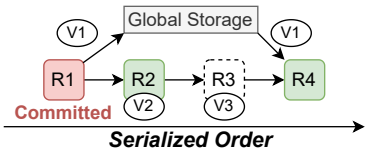


Figure 4. Application-level aborts can invalidate an inferred dependency order. R_1–R_4 are serialized by the dependency graph. R_1 writes data version V_1 to the storage. R_2 has not committed yet and buffers version V_2 locally.

the control plane, reconciliation repeatedly pays this scheduling and routing cost. Moreover, conflicts do not always affect the whole workflow. Retrying the entire workflow, as in previous systems [49, 59], repeats functions whose outputs remain valid, wasting computation and masking the function-level structure exposed by serverless workflows.

Solution: Function-level re-execution and fast path. During the retry phase, FaaSRep marks a function as dirty if it read expired data, and propagates this status along intra-workflow dependencies. A function remains clean only if its own reads are unchanged and all of its predecessors are also clean. FaaSRep therefore re-executes only dirty functions and reuses the previous results of clean ones. It also uses placement information from the initial execution to build a peer-to-peer communication fast path between containers, avoiding repeated control plane routing. Since the network endpoints are already known, this fast path avoids the handshake overhead seen in similar peer-to-peer approaches [33].

3 Overview of FaaSRep

Figure 5 shows the architecture of FaaSRep, a transactional runtime for serverless workflows that realizes workflow reconciliation to provide strict serializability.

The initial execution of FaaSRep leverages the common setting of optimistic protocols [2, 29]. The control plane schedules workflow functions across worker nodes, while each function accesses external transactional state through a lightweight sidecar that provides `read(key)`, `write(key, value)`, and `abort()` APIs. The sidecar reads from the local cache, buffers writes until commit, and records function-level metadata, including read/write sets and container placement. As the workflow proceeds, this metadata propagates along the workflow DAG and is collected by the local workflow sink. When a request finishes, the sink batches the complete metadata of multiple requests and submits it to the validator.

During validation, besides checking stale accesses, the FaaSRep validator also constructs dependencies for the batch at the same time (① & §4.1). After validation, the sidecar organizes workflow retries at function granularity. It bypasses the control plane through the inter-container fast path (② & §5.3) and re-executes only dirty functions (③ & §5.2). Each dirty function reads the correct data version as specified by

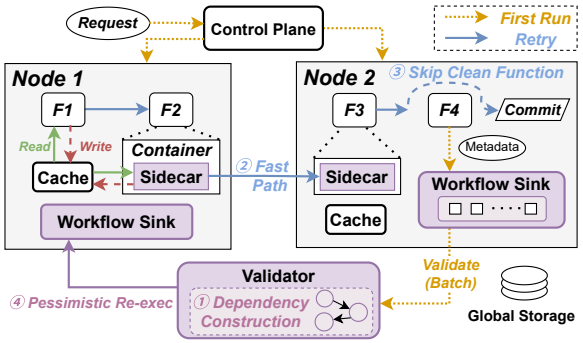


Figure 5. FaaSRep architecture. The workflow $f_1 \rightarrow f_2 \rightarrow f_3 \rightarrow f_4$ spans multiple worker nodes. Each worker maintains a local cache for external transactional state, whose entries may be stale with respect to the versions in global storage.

its upstream dependencies in the graph. If a request aborts during retry, the FaaSRep validator and the workflow sink cooperate to convert the affected successors to pessimistic mode and re-execute them (④ & §4.2, §5.1).

Assumptions and scope. FaaSRep assumes that applications access external state through the provided data APIs and do not perform arbitrary external I/O. It targets workflows with stable access sets, where identical inputs lead to accesses to the same external state. For dynamic access sets, FaaSRep preserves correctness by admitting only clean requests to reconciliation (§ 6.3).

4 Workflow Reconciliation Protocol

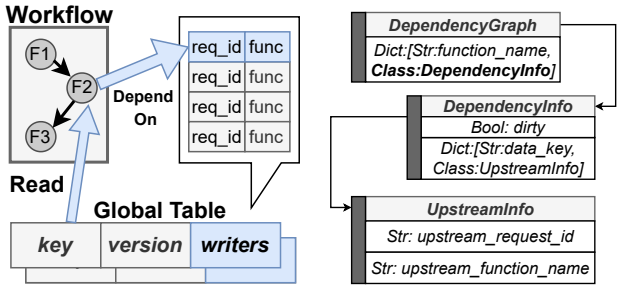
We describe low-overhead dependency construction during validation, explain the hybrid optimistic/pessimistic mode, and prove protocol correctness.

4.1 Delayed Dependency Construction

Workflow reconciliation begins by constructing a dependency graph. To avoid the communication overhead discussed in Section 2.4.1, FaaSRep defers construction until batch validation, reusing the collected access metadata.

In standard OCC [2, 19], the validator maintains a *global table* that records the latest version of each data key. During validation, the validator processes requests in the order in which they are added to the validation batch, which defines the *equivalent serial order* $\prec_B$ for each batch B: if request R_i appears before request R_j in B, then R_i precedes R_j in the serial history represented by this validation result. For each request, OCC checks its read set against the global table. A stale read causes the request to be discarded, while a valid request commits and updates the table with its write set.

FaaSRep preserves this structure, but extends each entry in the global table with a `writers` field recording uncommitted writers of the key, as shown in Figure 6a. These writers may come from earlier batches. For each read, FaaSRep first checks the corresponding `writers` field. If it is non-empty,



(a) New `writers` field. **(b)** Dependency Graph Structure.

Figure 6. Validator-side dependency construction and the dependency graph representation.

FaaSRep adds a dependency from the latest writer of this key to the reading function and marks the reader as `dirty`. Otherwise, FaaSRep performs the standard OCC version check against the global table. If the read version is stale, the reader is also marked as `dirty`, and the committed version recorded in the global table becomes its read source during retry. After processing the read sets of all functions in the request, FaaSRep appends their write sets to the corresponding `writers` fields. The dependency graph structure is shown in Figure 6b.

Besides cross-request dependencies, functions within the same request may form Read-Your-Writes (RYW) relationships [41, 47]. FaaSRep records this RYW information in workflow metadata during initial execution. After validation, these intra-workflow dependencies refine the dependency graph by overriding inter-request dependencies on the same data and propagating dirty marks along RYW edges.

For each of the n data accesses within a workflow, this dependency construction method only adds two extra constant-time operations: checking for the latest writer and updating the writers list. Consequently, the total overhead for building the dependency graph is $O(n)$, including the merge of RYW edges. This maintains the same asymptotic complexity as the standard OCC process [2].

4.2 Hybrid Reconciliation and Commit

After validation constructs the dependency graph, FaaSRep reconciles each batch with a hybrid optimistic/pessimistic protocol, as shown in Algorithm 1. To avoid the abort problem discussed in Section 2.4.2, FaaSRep uses a *readiness condition* to determine when a request can safely rely on its dependency graph. Intuitively, once this condition holds, the predecessors represented in the request's dependency graph will not later abort and invalidate the graph. The exact definition of this condition is given in Section 4.3.

The common case follows the optimistic path. For each validated batch B, FaaSRep uses the batch version V_B and the equivalent serial order $\prec_B$ produced by validation. Each request $R \in B$ starts in optimistic mode and retries according

Algorithm 1: Hybrid reconciliation protocol

Input: Validated batch B, batch version V_B, equivalent serial order $\prec_B$, dependency graph G_R for each request $R \in B$
Output: Batch B finishes consistently with $\prec_B$

```
1  foreach R ∈ B do
2      mode(R) ← optimistic; terminal(R) ← false;
3      readyToCommit(R) ← false; Retry(R, G_R);
4  while ∃R ∈ B : ¬terminal(R) do
5      foreach R ∈ B with ¬terminal(R) do
6          if ApplicationAbort(R) then
7              DiscardWrites(R); RemoveWriters(R);
8              terminal(R) ← true;
9          else if mode(R) = optimistic and
               PredAbortBeforeReady(R) then
10             DiscardRetryResult(R);
11             mode(R) ← pessimisticWait ;
12         else if mode(R) = pessimisticWait and
               Ready(R) then
13             G_R ← RebuildGraph(R);
14             mode(R) ← pessimisticRetry;
15             Retry(R, G_R) ;
16         else if Ready(R) and RetryDone(R) then
17             readyToCommit(R) ← true ;
18     if BatchReady(B) and
           PrecedingBatchesCommitted(B) then
19         W_B ← MergeLatestWrites(B, ≺_B) ;
20         PublishToGlobalStorage(W_B, V_B) ;
21         FlushCache(W_B, V_B) ;
22         UpdateGlobalTable(W_B, V_B) ;
23         RemoveBatchWriters(B) ;
24         foreach R ∈ B with readyToCommit(R) do
25             terminal(R) ← true;
```

to the dependency graph G_R constructed under $\prec_B$ (Lines 1–3). However, R cannot commit immediately after retry. It becomes ready to commit only after its retry completes and its readiness condition holds (Line 17).

If a predecessor of R aborts before R satisfies the readiness condition, the optimistic result of R may have been computed under an invalid dependency graph. FaaSRep then switches R to the pessimistic path and discards the optimistic retry result (Lines 9–11). Once R becomes ready, FaaSRep rebuilds G_R by excluding aborted predecessors and retries R under the rebuilt graph (Lines 12–15). Since this graph is rebuilt only after readiness holds, its represented predecessors will not later trigger application-level aborts and invalidate it. Therefore, pessimistic retry is performed at most once.

A batch becomes ready to commit when every request in it has reached a fixed outcome: aborted or ready to commit. FaaSRep does not publish a request as soon as it becomes ready. Instead, it commits the whole batch after all preceding batches have committed (Line 18), preserving the global serial order across batches and preventing a later batch from becoming visible before an earlier one.

During commit, FaaSRep merges the write sets of all non-aborted requests in B according to $\prec_B$ (Line 19). If multiple requests in the batch write the same key, the merged write set W_B keeps only the latest write under $\prec_B$. This is equivalent to executing the requests in $\prec_B$ order and publishing the final value of each key after the batch. FaaSRep then writes W_B to global storage with version V_B, refreshes the cache with the same version, and updates the global table used by later validation (Lines 20–22). Both commit and abort clear the corresponding `writers` entries (Lines 7 and 23), preventing terminal requests from appearing in future dependencies.

4.3 Readiness Condition

FaaSRep tries to determine as early as possible when a request can safely rely on its dependency graph. The condition should satisfy two requirements. First, it must ensure that the graph will not be invalidated by later application-level aborts. Second, it should be easy to track and should bound the cost of graph reconstruction in the pessimistic path. FaaSRep achieves this by defining readiness with one batch-level boundary and one request-level boundary.

The batch-level boundary limits cross-batch dependencies. For a request R in batch B, $\mathrm{PBD}(R)$ (pessimistic batch dependency) denotes the latest earlier batch that contains an upstream dependency of R. Instead of tracking every individual predecessor outside B, FaaSRep waits until $\mathrm{PBD}(R)$ has committed. Since batches commit in order, all earlier batches have also committed at that point. Therefore, cross-batch predecessors will not appear in a rebuilt dependency graph.

The request-level boundary limits dependencies inside the current batch. Let $\mathrm{PTD}(R)$ (pessimistic transaction dependency) denote the latest upstream dependency request of R in the equivalent serial order $\prec_B$. FaaSRep waits until every request up to $\mathrm{PTD}(R)$ in $\prec_B$ is ready to commit or has become terminal, forming a stable set of in-batch dependencies.

Formally, following the definition in Algorithm 1, R satisfies $\mathsf{Ready}(R)$ when both conditions hold:

$$\mathrm{PBD}(R) \text{ has committed, and}$$
$$\forall R' \preceq_B \mathrm{PTD}(R),\ \mathsf{readyToCommit}(R') \vee \mathsf{terminal}(R').$$

This definition bounds dependency reconstruction. When R enters the pessimistic path, FaaSRep waits for $\mathsf{Ready}(R)$ and rebuilds G_R using only earlier requests in the same batch up to $\mathrm{PTD}(R)$. Earlier batches are represented by committed versions in the global table, and later in-batch requests cannot precede R under $\prec_B$. Thus, readiness prevents the rebuilt graph from being invalidated again while avoiding unnecessary serialization of independent predecessors.

4.4 Correctness Analysis

We prove strict serializability for committed workflow requests with stable access sets. Requests with changing access sets are handled as discussed in Section 6.3. Aborted requests discard all buffered writes and are excluded from the committed history. Discarded retry results are also excluded.

For each batch B, we fix a request order $\prec_B$. Let S concatenate committed requests in batch commit order, following $\prec_B$ within each batch. The proof relies on three invariants.

Invariant 1: Fixed serial order. Retries, mode switches, and aborts preserve the relative order of committed requests. The validator assigns $\prec_B$ once during validation, and neither re-execution nor removal of aborted requests changes this order. Batches commit sequentially, and each batch merges writes according to $\prec_B$, retaining the last write to each key. Thus, S is well defined, and final write order is preserved.

Invariant 2: Predecessor stability. Dependencies of a committed request cannot be invalidated by later application-level aborts. A request R becomes ready to commit only when $\mathsf{Ready}(R)$ holds. At that point, batches through $\mathrm{PBD}(R)$ have committed, and requests through $\mathrm{PTD}(R)$ have either aborted or become ready to commit. Their outcomes and writes are therefore fixed. If a predecessor aborts before readiness, FaaSRep discards R's optimistic retry result and switches R to pessimistic mode. It then waits for readiness, rebuilds the graph without aborted predecessors, and retries. Hence, an invalidated optimistic result cannot be committed, and a rebuilt graph uses only stable predecessors.

Invariant 3: Read-source preservation. Every read retained in a committed execution observes the latest preceding write to its key in S, or the initial version if no such write exists. Intra-request dependencies preserve read-your-writes order. For other reads, validation identifies the latest preceding uncommitted writer, including writers from earlier batches, or checks the read against the committed version. A stale reader is marked dirty and re-executed using the recorded source. Dirty marks propagate to dependent functions, so reused results have unchanged inputs. If an abort changes the required source, Invariant 2 ensures that the invalid result is discarded and the source is recomputed after predecessors stabilize. Inductively along these dependencies, each retained read therefore observes its source in S.

Theorem. FaaSRep guarantees strict serializability for committed workflow requests within the stated scope.

Proof. Invariants 1–3 ensure that committed requests observe the same reads and produce the same final state as serial execution in S. It remains to show that S respects real-time order. If R_1 commits before R_2 begins, they cannot belong

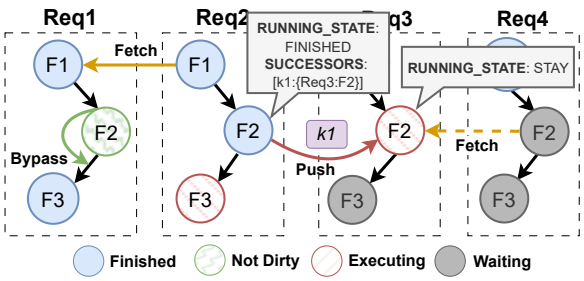


Figure 7. Function-level re-execution procedure. Req1.F2 is clean and is therefore skipped. Req2.F2 pushes the required data to its downstream successor, while Req4.F2 waits until Req3.F2 finishes before re-executing.

to the same batch: all requests in a batch finish execution before its publication. Since batches commit sequentially, R_2 must belong to a later batch and therefore follows R_1 in S. Thus, S preserves the real-time order of non-overlapping requests, establishing strict serializability.

5 Runtime Design of FaaSRep

Section 4.2 describes the reconciliation protocol. This section introduces how the runtime executes this protocol efficiently.

5.1 Runtime Coordination of Hybrid Reconciliation

The protocol defines several state transitions during reconciliation, including the reconciliation mode and readiness condition of each request. FaaSRep relies on the workflow sink to maintain these states during execution. The sink that submits a validation batch tracks the unfinished requests in the batch and coordinates their progress with the validator.

When retry starts, the validator returns the dependency graph to the control plane and registers it with the corresponding sink. The sink then uses it to update the successor information of unfinished requests. These successor relations allow the sink to locate the requests whose mode must change when a predecessor aborts, and to propagate the corresponding state update to them.

The sink also keeps a compact record for each unfinished request, recording its completion status and which mode currently governs it. Separately, the sink evaluates readiness for waiting requests from the completion states of related *PBD* and *PTD* dependencies. For requests in pessimistic mode, once readiness holds, the sink notifies the validator to reconstruct the dependency graph and start the pessimistic retry. When every request in the batch is either aborted or ready to commit, the sink asks the validator to commit the batch, keeping validation, retry, and commit progress consistent.

5.2 Retry at Function Granularity

FaaSRep retries a request at function granularity to match the serverless execution model [30], as shown in Figure 7.

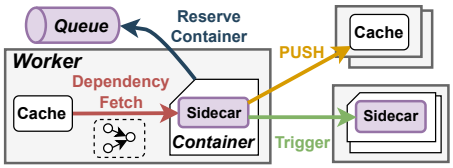


Figure 8. Fast path for reconciliation. During retry, sidecars fetch the dependency graph, trigger successor functions, and push required data to downstream function containers.

Before retrying a function, the control plane checks its `dirty` flag in the dependency graph. A clean function is skipped and reuses its initial result. A dirty function is re-executed after the control plane prepares its dependencies using the corresponding `DependencyInfo` described in Section 4.1.

`DependencyInfo` records the read sources required by each dirty function. Before re-execution, the control plane uses this metadata to fetch the required data versions from the cache on the corresponding worker nodes or from global storage. The function still invokes normal data APIs during re-execution, while the sidecar intercepts these calls and serves reads from the prefetched data. In this way, the user function observes the correct version transparently.

A dirty function may start execution before its upstream functions have produced new outputs. FaaSRep therefore maintains `RUNNING_STATE` and `SUCCESSORS` for each function. `RUNNING_STATE` records whether the function has produced consumable outputs, and `SUCCESSORS` records downstream functions waiting for the output. If an upstream function is not ready, the current function registers itself in the upstream function's `SUCCESSORS` and waits. When the upstream function finishes, it updates `RUNNING_STATE` and pushes the required data to the waiting successors. A skipped clean function performs the same notification because its reused output may also unblock downstream functions.

5.3 Fast Path for Function Re-execution

In the standard serverless workflow model, each function invocation still passes through the control plane before reaching its target container. During reconciliation, this path can offset the gain from fine-grained retry: even if FaaSRep re-executes only a subset of functions, each re-execution may still incur scheduling and forwarding overhead [33].

FaaSRep therefore introduces a **fast path** for the retry phase, as illustrated in Figure 8. After the initial execution, the control plane keeps the used containers in a dedicated queue to prevent them from being reclaimed [34]. It also collects the network endpoints of their sidecars and attaches this placement information to the metadata submitted for validation. After validation, the validator combines the placement information with the dependency graph and installs the graph into the cache of the corresponding worker nodes.

During retry, a sidecar reads the locally cached dependencies to identify upstream functions, required data items, and target sidecar endpoints. It then directly fetches data from upstream nodes, receives outputs pushed by completed predecessors, or wakes downstream containers through successor relations. This container-to-container path avoids repeated control plane mediation on the critical path and reduces each communication step to a direct transfer between the involved nodes. Large-scale deployments can further benefit from low-latency networking such as RDMA [24, 28].

6 Implementation

FaaSRep is implemented on top of FaaSFlow [31], whose distributed workflow manager reduces cross-node traffic. We extend FaaSFlow's workflow state to include each function's read/write sets and the network endpoints of its containers.

Our implementation uses two communication paths. The initial run follows the normal invocation path, where the control plane triggers functions through HTTP REST APIs. In contrast, the fast path for the retry phase is implemented as a peer-to-peer network using gRPC [51]. To ensure security, the gRPC fast path restricts communication to containers within the same workflow. Each function instance runs in an isolated Docker [18] container, with a sidecar packed in the image. This sidecar acts as both a gRPC server and client.

Each worker node hosts a Redis instance, partitioned to serve as a local cache for data, function states, and buffered writes. Functions access the Redis [43] instance and global database through their sidecar. The checking and updating of `RUNNING_STATE` and `SUCCESSORS` are implemented using Redis atomic operations to ensure correctness. The validator is implemented as a scalable `gevent` [23] WSGI server combined with a process pool to handle concurrent requests.

The cache maintains multiple versions only on the retry path. Normal execution reads the latest entry, while retry fetches the version specified in the dependency graph from global storage or the upstream worker node. Pinned versions are released after all dependent dirty functions complete.

6.1 Scalable Validator

FaaSRep decouples the validator to prevent it from becoming a scalability bottleneck in high-concurrency environments. Prior work [45, 49] reports little cross-application data sharing in the studied workload, with 99.7% of accessed data objects not shared across applications. Consequently, FaaSRep constructs validators at the application granularity, providing each application with its own dedicated validator and workflow sink instance. This model can seamlessly scale up or down as the number of applications changes. However, even a dedicated validator is still limited by its single-instance nature. To address this, FaaSRep further decomposes each validator into a single-instance *serializer* and multiple scalable *subworkers*. A proxy distributes incoming batch validation requests across the subworkers.

The serializer maintains the global table mentioned in Section 4.1, sequentially checks versions and finds dependencies. Multiple subworkers concurrently receive batches and prepare preliminary data, interacting with the serializer via a work queue. After receiving results from the serializer, they proceed with subsequent operations such as constructing the graph and sending retry requests. In the end, the serializer follows the order of `writers` to commit batches.

6.2 Failure Recovery

For the storage layer, FaaSRep relies on the underlying database's own fault tolerance mechanisms [44]. Here, we focus on failures of compute nodes and the FaaSRep validator.

At runtime, FaaSRep continuously monitors the status of each node. If a crash is detected, the validator discards all uncommitted requests, including those currently being retried or awaiting validation, and retries them after recovery. Requests that have already been committed are unaffected, as their modifications have been fully recorded.

FaaSRep uses the underlying database's transaction semantics to atomically commit all items in a batch's merged write set. However, a failure may interrupt subsequent updates to the validator's global table and related metadata. To support recovery, FaaSRep records each batch's modifications in the database as a *delta set* before committing and writes a completion log only after the database commit and all associated updates have finished.

When a validator crashes, all validation and retry operations of the corresponding application are paused. During recovery, the validator contacts the gateway to identify outstanding requests and checks the completion logs of their batches. For any batch without a completion log, FaaSRep uses its delta set to roll back any committed database changes. The validator then reconstructs its global table and restores the related metadata before resuming validation and retries.

6.3 Handling Dynamic Transactions

The scope in Section 3 covers representative transactional workflows, such as transfers [62] that access the source and destination accounts, and orders [4, 53] that access the corresponding inventory records. In these cases, the access set is discovered during execution and remains stable across retries of the same request, resembling the access stability assumed by prior database transaction systems [15, 19, 57].

Some workflows may violate this property. For example, a function may perform data-dependent accesses or enter a different branch. When FaaSRep detects such a change, it treats the dynamic request as an application-level abort. The request is then restarted, but joins reconciliation only if all its functions are clean. Otherwise, FaaSRep removes it from the validation batch and reruns it, like OCC. Since reconciliation retry skips clean functions, an admitted dynamic

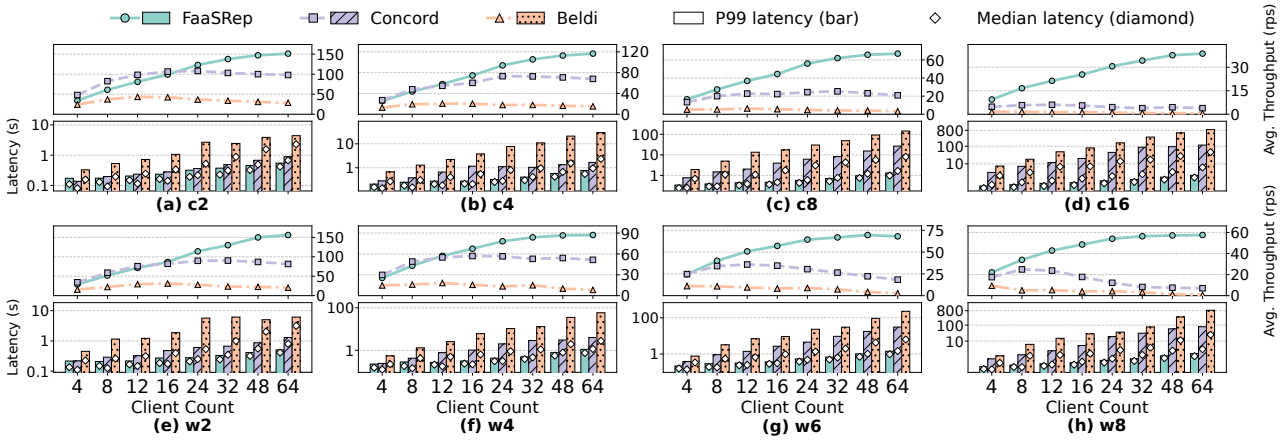


Figure 9. P99/P50 latency and average throughput for workflows with different lengths and fan-out degrees.

request does not issue changed accesses which invalidate the dependencies. This preserves correctness, but frequent changes may make FaaSRep behave similarly to OCC.

7 Evaluation

This section evaluates FaaSRep across workloads, contention levels, and deployment configurations. It first measures end-to-end performance, then isolates the contribution of each major component, analyzes sensitivity to system parameters, and finally quantifies runtime overhead and scalability.

7.1 Experimental Setup

All experiments run on an 8-node cluster with one storage node and seven worker nodes. Each node has a 32-core Intel Xeon Platinum 8562Y+ CPU and 256 GB memory. Docker containers are capped at 1 CPU core and 256 MB memory. The storage node hosts the gateway and FaaSRep validator. We use ScyllaDB [44] as the global storage backend for data, metadata, and logs, with strong-consistency reads enabled.

We use Beldi [59] and Concord [49] as main baselines. Beldi uses 2PL with remote locks and Wait-Die deadlock prevention, while Concord uses a cache coherence protocol. Both support concurrent reads through read-write locking. We also implement an OCC variant that represents FaaSRep without reconciliation. Boki's workflow library, BokiFlow [26], also provides strict serializability using Beldi's locking protocol over shared logs. Our BokiFlow implementation co-locates the shared-log service and database on a single node, which differs from the main deployment. We therefore evaluate it separately in Section 7.6. For fairness, all systems share the same FaaSFlow [31] codebase.

We evaluate two classes of workloads: microbenchmarks and real-world applications. The microbenchmarks comprise chain workflows of lengths 2, 4, 8, and 16 ($c2$, $c4$, $c8$, and $c16$) and fan-out workflows of degrees 2, 4, 6, and 8 ($w2$, $w4$, $w6$, and $w8$). These workflows access 4 KB objects across 10,000 keys, with keys selected according to a Zipf distribution [7]. We adopt three real-world applications from prior work. **Travel Reservation (Travel)** [16, 26, 59] reserves flights and cars before confirming payment and may abort if insufficient resources are available. **Banking System (Bank)** [21, 62] transfers money between two accounts. **Social Media (Social)** [16, 26, 59] allows users to atomically post comments and update timelines.

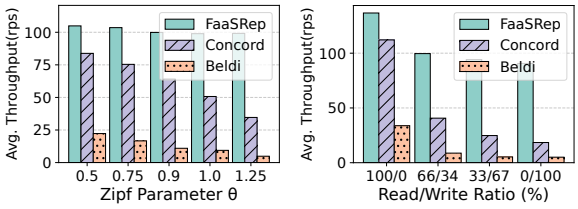
We first use closed-loop microbenchmarks to analyze FaaSRep across different workflow and access patterns, then evaluate real-world applications in both closed-loop tests and open-loop tests driven by Azure traces [45, 61]. We select two one-hour trace segments from the original dataset, representing high load (2k–2.5k RPM) and low load (0.5k–1.25k RPM). Since traces lack data-access information, they cannot capture conflict distributions. Our microbenchmarks cover a broad range of conflict intensities in Section 7.3.

7.2 Performance Across Workflow Structures

This experiment evaluates FaaSRep across different application styles using our microbenchmarks. We set the Zipf factor to 0.9, following mainstream configurations [14, 40, 52], and use a read/write ratio of 2:1 to model a hybrid pattern.

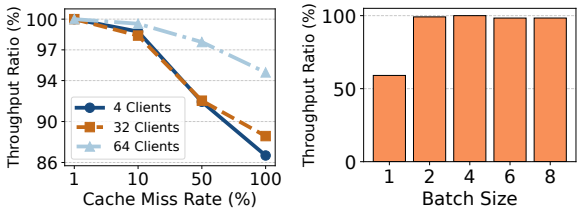
Figure 9 shows that FaaSRep outperforms both baselines in most settings. With 32 clients, FaaSRep achieves 1.33–8.81× (4.06–41.35×) higher throughput than Concord (Beldi). Its median latency is sometimes close to Concord, since both systems benefit from the cache. The main difference lies in tail latency: FaaSRep reduces P99 latency by more than 23.58% over Concord and 84.72% over Beldi, because reconciliation avoids repeated retry cycles after conflicts.

The performance gap becomes larger as the client count or workflow scale increases. Longer chains waste more computation upon aborts, and high fan-out workflows create more concurrent accesses. As a result, Beldi and Concord degrade in both throughput and latency. In the extreme case of $c16$



(a) Data skewness levels. **(b)** Read/write ratios.

Figure 10. Performance under varying data contention. A higher Zipf parameter indicates more skewed accesses.



(a) Impact of caching. **(b)** Impact of batch size.

Figure 11. Effects of different caching and batching settings.

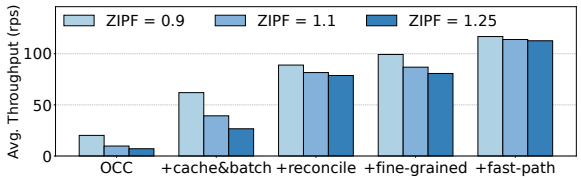


Figure 12. Design breakdown of FaaSRep. Each "+" label adds the corresponding component to the previous configuration.

with 64 clients, both baselines deliver less than 4 rps, while FaaSRep remains effective across all settings. Beldi performs worst, mainly because its remote-locking mechanism adds communication overhead to each data access.

FaaSRep also shows a trade-off in simple low-contention cases. In the *c*2 workflow with 4 clients, its throughput is 0.71× that of Concord. In such a rare-conflict setting, the fixed cost of validation and reconciliation is harder to amortize. This scenario therefore favors a simpler protocol.

7.3 Performance Under Varying Data Contention

We vary the Zipf factor θ from 0.5 to 1.25 and the read/write ratio from read-only to write-only. The conflict-affected rate is the fraction of baseline requests requiring multiple execution rounds because of conflicts. This and subsequent closed-loop tests use 32 clients as a moderate load. Sections 7.3–7.5 use the *c*4 microbenchmark workflow.

Figure 10a shows the impact of data skewness. As θ increases, accesses concentrate on fewer hot keys, causing the baselines to suffer more failed attempts. The conflict-affected rate rises from 9.06% to 64.72% in Concord and from 10.91% to 76.41% in Beldi. Concord remains competitive under low skew because of cached reads, but its throughput drops quickly when hot keys dominate the workload. In contrast, FaaSRep keeps its throughput stable, decreasing only from 104.9 to 99.14 rps. The reason is that the retry cost of FaaSRep is determined by how many functions become dirty. Retrying a dirty function incurs similar overhead regardless of whether all or only part of its data accesses conflict.

Figure 10b shows the effect of read/write ratios. Read-only workloads are contention-free, while the conflict-affected rate rises sharply as writes are introduced. For Concord, it reaches 54.59% and 61.81% at read ratios of 0.66 and 0.33, respectively. For Beldi, it reaches 64.69% and 75.48%. FaaSRep performs best even for read-only workloads, as its batch validation and commit amortize communication overhead.

7.4 Sensitivity Analysis

FaaSRep builds on OCC with caching and batching. We vary their configurations to evaluate their impact.

First, we control the cache miss rate for each data access, and calculate the throughput degradation compared to the case with a 1% miss rate. Figure 11a shows the results. Throughput drops as the miss rate increases, but by no more than 13.3% in all settings. These results indicate that FaaSRep's performance gains stem primarily from its effective reconciliation mechanism rather than caching.

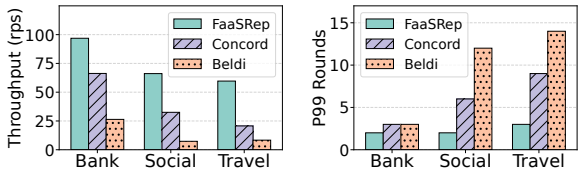
For batching, we change the batch size from 1 to 8, and measure the throughput ratio compared to the best case. Figure 11b shows that the main improvement occurs when the batch size increases to 2, with an increase of 67.9%. The subsequent growth shows a diminishing marginal effect, as a larger batch is harder to fill, delaying validation.

7.5 Ablation Study

Figure 12 illustrates the impact of FaaSRep's key design components under varying contention levels. When no additional mechanisms are applied, FaaSRep degrades to OCC. Enabling caching and batching can improve performance by 207.56–303.25%. The reconciliation mechanism yields further throughput gains ranging from 43.51% to 195.63%. This benefit becomes especially pronounced as contention intensifies, which can be properly addressed by FaaSRep.

Fine-grained retry (§5.2) adds a 2.52–11.66% improvement. Low-contention scenarios benefit more because more functions remain unaffected by conflicts and can skip retry, reducing redundant computation.

Finally, when the fast path optimization (§5.3) is enabled, the throughput increases by another 17.64–39.6%, showing the effectiveness of bypassing the control plane. The benefit is also more noticeable under high contention settings, which introduce heavier loads to the control plane.



(a) Throughput. **(b)** Execution rounds.

Figure 13. Throughput and P99 execution rounds of three real-world applications in the closed-loop test.

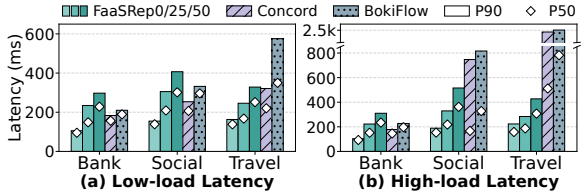


Figure 14. Application latencies in the trace-driven open-loop test. FaaSRep0/25/50 denotes application-level abort rates of 0%, 25%, and 50% during retry, respectively.

7.6 Performance in Real-world Scenarios

We evaluate FaaSRep with the three real-world applications described in Section 7.1. This experiment includes a closed-loop test for throughput and an open-loop test driven by production-level traces. Some requests may proactively trigger aborts when application conditions are not satisfied, such as insufficient balance or inventory.

Figure 13 reports the closed-loop results. FaaSRep performs best across all applications, improving throughput by 3.81–9.01× over Beldi and 1.44–2.73× over Concord. These gains are not limited to high contention cases. Using the conflict-affected rate defined in Section 7.3, Concord observes 10.84%, 40.94%, and 50.21% conflict-affected requests for Bank, Social, and Travel, respectively. Bank suffers the least from conflicts, leaving FaaSRep the smallest room for improvement. Figure 13b further shows that FaaSRep keeps retry cost bounded. While the baselines may require many rounds to resolve conflicts, FaaSRep completes each request within at most three rounds, including cases that fall back to pessimistic reconciliation after application-level aborts.

Figure 14 reports P50 and P90 latencies under the two Azure traces in Section 7.1. Since these traces do not capture application-level abort patterns, we vary the probability of application-level abort during retry across 0%, 25%, and 50%. Without application-level aborts, FaaSRep outperforms both Concord and BokiFlow across all applications and load levels. Higher abort probabilities increase its latency by triggering more pessimistic retries. Under low load, FaaSRep at 25% and 50% abort probabilities can be slower than the baselines but remains competitive. Under high load, both baselines suffer sharp tail-latency growth for Social and Travel, and even fail to keep latency bounded for Travel. In contrast, FaaSRep outperforms both baselines on these two applications even at a 50% abort probability.

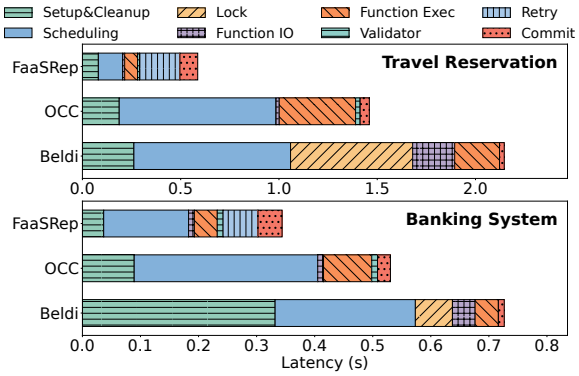


Figure 15. Execution time breakdown for Travel (top, high contention) and Bank (bottom, low contention).

7.7 Execution Time Breakdown

This experiment breaks down the average latency in the closed-loop setting to explain where the performance gap comes from. We use Travel as a high-contention workload and Banking as a lower-contention workload.

In the top half of Figure 15, OCC's caching reduces function IO latency relative to Beldi, but contention-based retries impose high scheduling and function execution costs on both. Beldi's remote locking costs exceed function execution time, and lock-entry cleanup adds substantial latency. FaaSRep's savings from eliminating repeated retries far outweigh its reconciliation and sequential commit costs.

Under low contention, OCC and Beldi waste fewer attempts. FaaSRep still achieves lower latency with less reconciliation work, but protocol overhead, such as sequential commit, becomes more visible. For extremely simple workflows with rare conflicts, fixed reconciliation costs are harder to amortize, narrowing the performance gap.

7.8 Overhead and Scalability

We first evaluate the overhead of FaaSRep. We compare it with OCC while co-locating the three real-world applications, and use CPU usage rather than CPU time as the CPU metric. Since the validator runs on the storage backend, we exclude it here and evaluate it separately below. As shown in Figure 16, FaaSRep increases module CPU and memory usage by only 4.1% and 0.5%, respectively. Bandwidth increases by 41.5%, but the absolute increase is only 1278.83 KB/s, mainly from fast path communication and retry metadata exchange. The sidecar increases container memory usage by only 4.2%, so user-visible monetary cost does not increase in practice.

We next evaluate scalability. FaaSRep uses the same function execution framework as prior serverless systems [31,

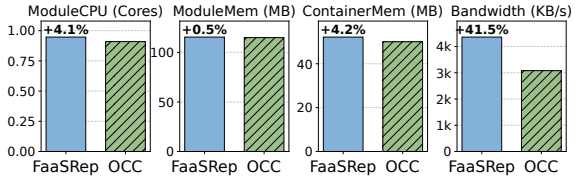


Figure 16. FaaSRep overhead, including module CPU and memory usage, container memory usage, and inter-worker network bandwidth.

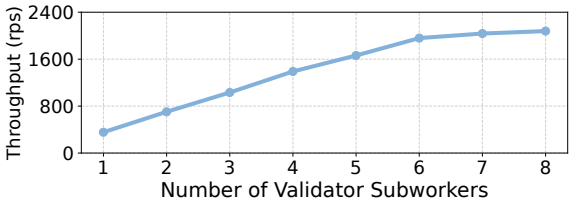


Figure 17. FaaSRep validator processing capability with different numbers of subworkers.

58, 59], so the main new scalability concern is the validator per application. The validator includes a sequential stage for validation and commit, but most preprocessing and metadata generation can be parallelized across subworkers. We therefore measure the maximum validation throughput while varying the number of validator subworkers, sending batches of the Travel application with a batch size of 4.

Figure 17 shows that the validator throughput increases with the number of subworkers and saturates at eight subworkers, reaching 2078.47 rps. This capacity is higher than the peak request rate of many real-world serverless traces, such as the Azure Function trace [45], whose peak is no more than 500 rps. It is also above the default concurrency limit of common commercial platforms, such as the 1000 concurrent-function limit per AWS Lambda account [5].

8 Related Work

Transactional support in serverless computing with strong consistency. Beldi [59] implements a Two-Phase Locking (2PL) protocol to ensure strict serializability, maintaining locks in a global storage service. For efficiency, Concord [49] utilizes a directory-based cache coherence protocol, while Boki [26] and Halfmoon [42] record lock entries in a shared log abstraction, reducing the overhead of remote lock accesses in Beldi. Another work, RTSFaaS [62], exploits a lease-based concurrency control protocol, but requires a priori knowledge of the entire access set to build its task-precedence graph.

Serverless transactions with weak consistency. Several systems use weak consistency to avoid the overhead of strong consistency [37, 47, 48, 56, 60]. AFT [47] deploys a fault-tolerance shim between the computing and storage layers, offering *Read Atomicity* [8] guarantees. CloudBurst [48] and FaaSTCC [37] offer *Transactional Causal Consistency* guarantees by checking whether the causal dependency is violated during runtime. All these approaches do not prevent concurrent write conflicts, thereby violating serializability.

Concurrency control in database systems. Prior work improves transaction execution in both single-node [13, 25, 50, 55] and distributed databases [36, 52, 54]. Bamboo [25] increases concurrency by early lock releasing, while Calvin [52] avoids distributed coordination by ordering transactions before execution. These techniques motivate efficient concurrency control but do not target transactional workflows that span multiple independently scheduled serverless functions.

Transaction re-execution in database systems. Transaction healing and repair [15, 57] use static analysis to identify dependencies among data-access operations within a transaction. Hackwrench [19] locally tracks dependencies within a batch and serializes retries within each conflicting sub-batch while holding locks until commit, causing blocking. Aria [35] switches between OCC-style retries in later batches and an ordered locking fallback that blocks conflicting successors until commit. Morty [10] discovers object-level conflicts online and uses continuations to rewind affected computations, potentially re-executing a transaction multiple times. FaaSRep uses a global function-level dependency graph to coordinate an organized retry, pipelining affected functions across batches as soon as buffered outputs become available, without waiting for predecessor batches to commit. Its optimistic and pessimistic reconciliation modes coexist at request granularity: only requests with invalidated dependencies enter the fallback, bounding the retry rounds.

9 Conclusion

FaaSRep provides strictly serializable transactions for serverless workflows with high performance. Workflow reconciliation uses conflicts discovered during initial execution to schedule retry through function-level dependencies. Dependency construction at validation time avoids runtime overhead, while a hybrid protocol correctly handles application-level aborts. Function-level retry and a peer-to-peer fast path minimize control plane involvement. Evaluation shows significantly higher throughput and lower response latency than existing solutions.

Acknowledgments

We thank the reviewers and our shepherd for their suggestions. This work is partially sponsored by the National Natural Science Foundation of China (62232011, 62302302). Zijun Li, Quan Chen and Jie Wu are the corresponding authors.

References

[1] Mainak Adhikari, Tarachand Amgoth, and Satish Narayana Srirama. 2019. A Survey on Scheduling Strategies for Workflows in Cloud Environment and Emerging Trends. *Comput. Surveys* 52, 4, Article 68 (2019), 36 pages. doi:10.1145/3325097

[2] Marcos K Aguilera, Arif Merchant, Mehul Shah, Alistair Veitch, and Christos Karamanolis. 2007. Sinfonia: a new paradigm for building scalable distributed systems. *ACM SIGOPS Operating Systems Review* 41, 6 (2007), 159–174. doi:10.1145/1323293.1294278

[3] Ahsan Ali, Riccardo Pinciroli, Feng Yan, and Evgenia Smirni. 2020. BATCH: Machine Learning Inference Serving on Serverless Platforms with Adaptive Batching. In *SC20: International Conference for High Performance Computing, Networking, Storage and Analysis*. IEEE, 1–15. doi:10.1109/sc41405.2020.00073

[4] Alibaba Cloud. 2026. Serverless App Engine (SAE). Retrieved October 15, 2026 from https://www.aliyun.com/product/aliware/sae

[5] Amazon Web Services. 2026. AWS Lambda Quotas. Retrieved October 15, 2026 from https://docs.amazonaws.cn/lambda/latest/dg/gettingstarted-limits.html

[6] Lixiang Ao, Liz Izhikevich, Geoffrey M. Voelker, and George Porter. 2018. Sprocket: A Serverless Video Processing Framework. In *Proceedings of the ACM Symposium on Cloud Computing*. Association for Computing Machinery, New York, NY, USA, 263–274. doi:10.1145/3267809.3267815

[7] Robert L Axtell. 2001. Zipf Distribution of U.S. Firm Sizes. *Science* 293, 5536 (2001), 1818–1820. doi:10.1126/science.1062081

[8] Peter Bailis, Alan Fekete, Ali Ghodsi, Joseph M Hellerstein, and Ion Stoica. 2016. Scalable Atomic Visibility with RAMP Transactions. *ACM Transactions on Database Systems* 41, 3, Article 15 (2016), 45 pages. doi:10.1145/2909870

[9] Philip A. Bernstein, David W. Shipman, and Wing S. Wong. 1979. Formal Aspects of Serializability in Database Concurrency Control. *IEEE Transactions on Software Engineering* SE-5, 3 (1979), 203–216. doi:10.1109/tse.1979.234182

[10] Matthew Burke, Florian Suri-Payer, Jeffrey Helt, Lorenzo Alvisi, and Natacha Crooks. 2023. Morty: Scaling Concurrency Control with Re-Execution. In *Proceedings of the Eighteenth European Conference on Computer Systems (EuroSys '23)*. Association for Computing Machinery, New York, NY, USA, 687–702. doi:10.1145/3552326.3567500

[11] João Carreira, Pedro Fonseca, Alexey Tumanov, Andrew Zhang, and Randy H. Katz. 2019. Cirrus: a Serverless Framework for End-to-end ML Workflows. In *Proceedings of the ACM Symposium on Cloud Computing*. Association for Computing Machinery, New York, NY, USA, 13–24. doi:10.1145/3357223.3362711

[12] Benjamin Carver, Jingyuan Zhang, Ao Wang, Ali Anwar, Panruo Wu, and Yue Cheng. 2020. Wukong: a scalable and locality-enhanced framework for serverless parallel computing. In *Proceedings of the 11th ACM Symposium on Cloud Computing*. Association for Computing Machinery, New York, NY, USA, 1–15. doi:10.1145/3419111.3421286

[13] Youmin Chen, Xiangyao Yu, Paraschos Koutris, Andrea C Arpaci-Dusseau, Remzi H Arpaci-Dusseau, and Jiwu Shu. 2022. Plor: General Transactions with Predictable, Low Tail Latency. In *Proceedings of the 2022 International Conference on Management of Data*. Association for Computing Machinery, New York, NY, USA, 19–33. doi:10.1145/3514221.3517879

[14] Brian F Cooper, Adam Silberstein, Erwin Tam, Raghu Ramakrishnan, and Russell Sears. 2010. Benchmarking cloud serving systems with YCSB. In *Proceedings of the 1st ACM Symposium on Cloud Computing*. Association for Computing Machinery, New York, NY, USA, 143–154. doi:10.1145/1807128.1807152

[15] Mohammad Dashti, Sachin Basil John, Amir Shaikhha, and Christoph Koch. 2017. Transaction Repair for Multi-Version Concurrency Control. In *Proceedings of the 2017 ACM International Conference on Management of Data*. Association for Computing Machinery, New York, NY, USA, 235–250. doi:10.1145/3035918.3035919

[16] DeathStarBench Contributors. 2026. DeathStarBench: Open-source Benchmark Suite for Cloud Microservices. Retrieved October 15, 2026 from https://github.com/delimitrou/DeathStarBench/

[17] Anitha Deenadayalan. 2021. Building a Serverless Distributed Application Using a Saga Orchestration Pattern. AWS Compute Blog. Retrieved October 15, 2026 from https://aws.amazon.com/blogs/compute/building-a-serverless-distributed-application-using-a-saga-orchestration-pattern/

[18] Docker. 2026. Docker. Retrieved October 15, 2026 from https://www.docker.com/

[19] Zhiyuan Dong, Zhaoguo Wang, Xiaodong Zhang, Xian Xu, Changgeng Zhao, Haibo Chen, Aurojit Panda, and Jinyang Li. 2023. Fine-Grained Re-Execution for Efficient Batched Commit of Distributed Transactions. *Proceedings of the VLDB Endowment* 16, 8 (2023), 1930–1943. doi:10.14778/3594512.3594523

[20] Sadjad Fouladi, Francisco Romero, Dan Iter, Qian Li, Shuvo Chatterjee, Christos Kozyrakis, Matei Zaharia, and Keith Winstein. 2019. From Laptop to Lambda: Outsourcing Everyday Jobs to Thousands of Transient Functional Containers. In *2019 USENIX Annual Technical Conference (USENIX ATC 19)*. USENIX Association, 475–488. https://www.usenix.org/conference/atc19/presentation/fouladi

[21] Yu Gan, Yanqi Zhang, Dailun Cheng, Ankitha Shetty, Priyal Rathi, Nayan Katarki, Ariana Bruno, Justin Hu, Brian Ritchken, Brendon Jackson, Kelvin Hu, Meghna Pancholi, Yuan He, Brett Clancy, Chris Colen, Fukang Wen, Catherine Leung, Siyuan Wang, Leon Zaruvinsky, Mateo Espinosa, Rick Lin, Zhongling Liu, Jake Padilla, and Christina Delimitrou. 2019. An Open-Source Benchmark Suite for Microservices and Their Hardware-Software Implications for Cloud & Edge Systems. In *Proceedings of the Twenty-Fourth International Conference on Architectural Support for Programming Languages and Operating Systems*. Association for Computing Machinery, New York, NY, USA, 3–18. doi:10.1145/3297858.3304013

[22] Hector Garcia-Molina and Kenneth Salem. 1987. Sagas. In *Proceedings of the 1987 ACM SIGMOD international conference on Management of data - SIGMOD '87*. Association for Computing Machinery, New York, New York, USA, 249–259. doi:10.1145/38713.38742

[23] gevent Contributors. 2026. gevent: A Coroutine-based Python Networking Library. Retrieved October 15, 2026 from https://www.gevent.org/

[24] Chuanxiong Guo, Haitao Wu, Zhong Deng, Gaurav Soni, Jianxi Ye, Jitu Padhye, and Marina Lipshteyn. 2016. RDMA over Commodity Ethernet at Scale. In *Proceedings of the 2016 ACM SIGCOMM Conference*. Association for Computing Machinery, New York, NY, USA, 202–215. doi:10.1145/2934872.2934908

[25] Zhihan Guo, Kan Wu, Cong Yan, and Xiangyao Yu. 2021. Releasing Locks As Early As You Can: Reducing Contention of Hotspots by Violating Two-Phase Locking. In *Proceedings of the 2021 International Conference on Management of Data*. Association for Computing Machinery, New York, NY, USA, 658–670. doi:10.1145/3448016.3457294

[26] Zhipeng Jia and Emmett Witchel. 2021. Boki: Stateful Serverless Computing with Shared Logs. In *Proceedings of the ACM SIGOPS 28th Symposium on Operating Systems Principles*. Association for Computing Machinery, New York, NY, USA, 691–707. doi:10.1145/3477132.3483541

[27] Eric Jonas, Johann Schleier-Smith, Vikram Sreekanti, Chia-Che Tsai, Anurag Khandelwal, Qifan Pu, Vaishaal Shankar, João Carreira, Karl Krauth, Neeraja Yadwadkar, Joseph E. Gonzalez, Raluca Ada Popa, Ion Stoica, and David A. Patterson. 2019. Cloud Programming Simplified: A Berkeley View on Serverless Computing. arXiv:1902.03383 [cs.OS] doi:10.48550/arXiv.1902.03383

[28] Gurkirat Kaur and Manju Bala. 2013. RDMA over Converged Ethernet: A Review. *International Journal of Advances in Engineering & Technology* 6, 4 (2013), 1890–1894. https://www.ijaet.org/media/51I16-IJAET0916934_v6_iss4_1890to1894.pdf

[29] Hsiang-Tsung Kung and John T Robinson. 1981. On optimistic methods for concurrency control. *ACM Transactions on Database Systems* 6, 2 (1981), 213–226. doi:10.1145/319566.319567

[30] Zijun Li, Linsong Guo, Jiagan Cheng, Quan Chen, Bingsheng He, and Minyi Guo. 2022. The Serverless Computing Survey: A Technical Primer for Design Architecture. *Comput. Surveys* 54, 10s, Article 220 (2022), 34 pages. doi:10.1145/3508360

[31] Zijun Li, Yushi Liu, Linsong Guo, Quan Chen, Jiagan Cheng, Wenli Zheng, and Minyi Guo. 2022. FaaSFlow: enable efficient workflow execution for function-as-a-service. In *Proceedings of the 27th ACM International Conference on Architectural Support for Programming Languages and Operating Systems*. Association for Computing Machinery, New York, NY, USA, 782–796. doi:10.1145/3503222.3507717

[32] Zijun Li, Chuhao Xu, Quan Chen, Jieru Zhao, Chen Chen, and Minyi Guo. 2023. DataFlower: Exploiting the Data-flow Paradigm for Serverless Workflow Orchestration. In *Proceedings of the 28th ACM International Conference on Architectural Support for Programming Languages and Operating Systems, Volume 4*. Association for Computing Machinery, New York, NY, USA, 57–72. doi:10.1145/3623278.3624755

[33] Guowei Liu, Laiping Zhao, Yiming Li, Zhaolin Duan, Sheng Chen, Yitao Hu, Zhiyuan Su, and Wenyu Qu. 2024. FUYAO: DPU-enabled Direct Data Transfer for Serverless Computing. In *Proceedings of the 29th ACM International Conference on Architectural Support for Programming Languages and Operating Systems, Volume 3*. Association for Computing Machinery, New York, NY, USA, 431–447. doi:10.1145/3620666.3651327

[34] Qichang Liu, Yue Cheng, Haiying Shen, Ao Wang, and Bharathan Balaji. 2025. Concurrency-Informed Orchestration for Serverless Functions. In *Proceedings of the 30th ACM International Conference on Architectural Support for Programming Languages and Operating Systems, Volume 2*. Association for Computing Machinery, New York, NY, USA, 147–161. doi:10.1145/3676641.3716253

[35] Yi Lu, Xiangyao Yu, Lei Cao, and Samuel Madden. 2020. Aria: a fast and practical deterministic OLTP database. *Proceedings of the VLDB Endowment* 13, 11 (2020), 2047–2060. doi:10.14778/3407790.3407808

[36] Yi Lu, Xiangyao Yu, Lei Cao, and Samuel Madden. 2021. Epoch-based commit and replication in distributed OLTP databases. *Proceedings of the VLDB Endowment* 14, 5 (2021), 743–756. doi:10.14778/3446095.3446098

[37] Taras Lykhenko, Rafael Soares, and Luis Rodrigues. 2021. FaaSTCC: efficient transactional causal consistency for serverless computing. In *Proceedings of the 22nd International Middleware Conference*. Association for Computing Machinery, New York, NY, USA, 159–171. doi:10.1145/3464298.3493392

[38] Ashraf Mahgoub, Karthick Shankar, Subrata Mitra, Ana Klimovic, Somali Chaterji, and Saurabh Bagchi. 2021. SONIC: Application-aware Data Passing for Chained Serverless Applications. In *2021 USENIX Annual Technical Conference (USENIX ATC 21)*. USENIX Association, 285–301. https://www.usenix.org/conference/atc21/presentation/mahgoub

[39] Ashraf Mahgoub, Edgardo Barsallo Yi, Karthick Shankar, Sameh Elnikety, Somali Chaterji, and Saurabh Bagchi. 2022. ORION and the Three Rights: Sizing, Bundling, and Prewarming for Serverless DAGs. In *16th USENIX Symposium on Operating Systems Design and Implementation (OSDI 22)*. USENIX Association, 303–320. https://www.usenix.org/conference/osdi22/presentation/mahgoub

[40] Yandong Mao, Eddie Kohler, and Robert Tappan Morris. 2012. Cache craftiness for fast multicore key-value storage. In *Proceedings of the 7th ACM European conference on Computer Systems*. Association for Computing Machinery, New York, NY, USA, 183–196. doi:10.1145/2168836.2168855

[41] Hesam Nejati Sharif Aldin, Hossein Deldari, Mohammad Hossein Moattar, and Mostafa Razavi Ghods. 2020. Strict Timed Causal Consistency as a Hybrid Consistency Model in the Cloud Environment. *Future Generation Computer Systems* 105 (2020), 259–274. doi:10.1016/j.future.2019.11.038

[42] Sheng Qi, Xuanzhe Liu, and Xin Jin. 2023. Halfmoon: Log-Optimal Fault-Tolerant Stateful Serverless Computing. In *Proceedings of the 29th Symposium on Operating Systems Principles*. Association for Computing Machinery, New York, NY, USA, 314–330. doi:10.1145/3600006.3613154

[43] Redis. 2026. Redis. Retrieved October 15, 2026 from https://redis.io/

[44] ScyllaDB. 2026. ScyllaDB. Retrieved October 15, 2026 from https://www.scylladb.com/

[45] Mohammad Shahrad, Rodrigo Fonseca, Inigo Goiri, Gohar Chaudhry, Paul Batum, Jason Cooke, Eduardo Laureano, Colby Tresness, Mark Russinovich, and Ricardo Bianchini. 2020. Serverless in the Wild: Characterizing and Optimizing the Serverless Workload at a Large Cloud Provider. In *2020 USENIX Annual Technical Conference (USENIX ATC 20)*. USENIX Association, 205–218. https://www.usenix.org/conference/atc20/presentation/shahrad

[46] Abraham Silberschatz, James L Peterson, and Peter B Galvin. 1991. *Operating System Concepts* (3 ed.). Addison-Wesley. https://ci.nii.ac.jp/ncid/BA12372447

[47] Vikram Sreekanti, Chenggang Wu, Saurav Chhatrapati, Joseph E Gonzalez, Joseph M Hellerstein, and Jose M Faleiro. 2020. A fault-tolerance shim for serverless computing. In *Proceedings of the Fifteenth European Conference on Computer Systems*. Association for Computing Machinery, New York, NY, USA, 1–15. doi:10.1145/3342195.3387535

[48] Vikram Sreekanti, Chenggang Wu, Xiayue Charles Lin, Johann Schleier-Smith, Joseph E. Gonzalez, Joseph M. Hellerstein, and Alexey Tumanov. 2020. Cloudburst: stateful functions-as-a-service. *Proceedings of the VLDB Endowment* 13, 11 (2020), 2438–2452. doi:10.14778/3407790.3407836

[49] Jovan Stojkovic, Chloe Alverti, Alan Andrade, Nikoleta Iliakopoulou, Hubertus Franke, Tianyin Xu, and Josep Torrellas. 2025. Concord: Rethinking Distributed Coherence for Software Caches in Serverless Environments. In *2025 IEEE International Symposium on High Performance Computer Architecture (HPCA)*. IEEE, 458–473. doi:10.1109/hpca61900.2025.00043

[50] Dixin Tang and Aaron J Elmore. 2018. Toward Coordination-free and Reconfigurable Mixed Concurrency Control. In *2018 USENIX Annual Technical Conference (USENIX ATC 18)*. USENIX Association, 809–822. https://www.usenix.org/conference/atc18/presentation/tang

[51] The gRPC Authors. 2026. gRPC: A High Performance, Open Source Universal RPC Framework. Retrieved October 15, 2026 from https://grpc.io/

[52] Alexander Thomson, Thaddeus Diamond, Shu-Chun Weng, Kun Ren, Philip Shao, and Daniel J Abadi. 2012. Calvin: fast distributed transactions for partitioned database systems. In *Proceedings of the 2012 ACM SIGMOD International Conference on Management of Data*. Association for Computing Machinery, New York, NY, USA, 1–12. doi:10.1145/2213836.2213838

[53] tySupermarket 2026. tySupermarket for Worry-free Purchasing. https://ztf58.cn/

[54] Alexandre Verbitski, Anurag Gupta, Debanjan Saha, Murali Brahmadesam, Kamal Gupta, Raman Mittal, Sailesh Krishnamurthy, Sandor Maurice, Tengiz Kharatishvili, and Xiaofeng Bao. 2017. Amazon Aurora: Design Considerations for High Throughput Cloud-Native Relational Databases. In *Proceedings of the 2017 ACM International Conference on Management of Data*. Association for Computing Machinery, New York, NY, USA, 1041–1052. doi:10.1145/3035918.3056101

[55] Tianzheng Wang and Hideaki Kimura. 2016. Mostly-optimistic concurrency control for highly contended dynamic workloads on a thousand cores. *Proceedings of the VLDB Endowment* 10, 2 (2016), 49–60. doi:10.14778/3015274.3015276

[56] Chenggang Wu, Vikram Sreekanti, and Joseph M Hellerstein. 2020. Transactional Causal Consistency for Serverless Computing. In *Proceedings of the 2020 ACM SIGMOD International Conference on Management of Data*. Association for Computing Machinery, New York, NY, USA, 83–97. doi:10.1145/3318464.3389710

[57] Yingjun Wu, Chee-Yong Chan, and Kian-Lee Tan. 2016. Transaction Healing: Scaling Optimistic Concurrency Control on Multicores. In *Proceedings of the 2016 International Conference on Management of Data*. Association for Computing Machinery, New York, NY, USA, 1689–1704. doi:10.1145/2882903.2915202

[58] Chuhao Xu, Yiyu Liu, Zijun Li, Quan Chen, Han Zhao, Deze Zeng, Qian Peng, Xueqi Wu, Haifeng Zhao, Senbo Fu, and Minyi Guo. 2024. FaaSMem: Improving Memory Efficiency of Serverless Computing with Memory Pool Architecture. In *Proceedings of the 29th ACM International Conference on Architectural Support for Programming Languages and Operating Systems, Volume 3*. Association for Computing Machinery, New York, NY, USA, 331–348. doi:10.1145/3620666.3651355

[59] Haoran Zhang, Adney Cardoza, Peter Baile Chen, Sebastian Angel, and Vincent Liu. 2020. Fault-tolerant and Transactional Stateful Serverless Workflows. In *14th USENIX Symposium on Operating Systems Design and Implementation (OSDI 20)*. USENIX Association, 1187–1204. https://www.usenix.org/conference/osdi20/presentation/zhang-haoran

[60] Haoran Zhang, Shuai Mu, Sebastian Angel, and Vincent Liu. 2025. CausalMesh: A Causal Cache for Stateful Serverless Computing. *ACM SIGMOD Record* 54, 1 (2025), 48–57. doi:10.1145/3733620.3733631

[61] Yanqi Zhang, Íñigo Goiri, Gohar Irfan Chaudhry, Rodrigo Fonseca, Sameh Elnikety, Christina Delimitrou, and Ricardo Bianchini. 2021. Faster and Cheaper Serverless Computing on Harvested Resources. In *Proceedings of the ACM SIGOPS 28th Symposium on Operating Systems Principles*. Association for Computing Machinery, New York, NY, USA, 724–739. doi:10.1145/3477132.3483580

[62] Jianjun Zhao, Haikun Liu, Shuhao Zhang, Haodi Lu, Yancan Mao, Zhuohui Duan, Xiaofei Liao, and Hai Jin. 2025. Towards High-Performance Transactional Stateful Serverless Workflows with Affinity-Aware Leasing. In *2025 USENIX Annual Technical Conference (USENIX ATC 25)*. USENIX Association, 1535–1551. https://www.usenix.org/conference/atc25/presentation/zhao-jianjun